# CSC3002
# C/C++ Programming

# Review – Part I
# Lectures 10-15

**Prof. Dr. Ruizhi Chen， SDS**

**Office: TxC 717**

**E-mail: chenruizhi@cuhk.edu.cn**

# Introduction to the C++ Standard Libraries

- A collection of ***classes*** and ***functions***, which are written in the core language and part of the C++ ISO Standard itself. Features of the C++ Standard Library are declared within the `std` ***namespace***
  - Containers: vector, queue, stack, map, set, etc.
  - General: algorithm, functional, iterator, memory, etc.
  - Strings
  - Streams and Input/Output: iostream, fstream, sstream, etc.
  - Localization
  - Language support
  - Thread support library
  - Numerics library
  - C standard library: cmath, cctype, cstring, cstdio, cstdlib, etc.

## Container class templates

**Sequence containers:**

| | |
|---|---|
| **array** C++11 | Array class (class template ) |
| **vector** | Vector (class template ) |
| **deque** | Double ended queue (class template ) |
| **forward_list** C++11 | Forward list (class template ) |
| **list** | List (class template ) |

**Container adaptors:**

| | |
|---|---|
| **stack** | LIFO stack (class template ) |
| **queue** | FIFO queue (class template ) |
| **priority_queue** | Priority queue (class template ) |

**Associative containers:**

| | |
|---|---|
| **set** | Set (class template ) |
| **multiset** | Multiple-key set (class template ) |
| **map** | Map (class template ) |
| **multimap** | Multiple-key map (class template ) |

**Unordered associative containers:**

| | |
|---|---|
| **unordered_set** C++11 | Unordered Set (class template ) |
| **unordered_multiset** C++11 | Unordered Multiset (class template ) |
| **unordered_map** C++11 | Unordered Map (class template ) |
| **unordered_multimap** C++11 | Unordered Multimap (class template ) |

**Other:**

Two class templates share certain properties with containers, and are sometimes classified with them: `bitset` and `valarray`.

# The `vector<`*type*`>` Class

- The **`vector`** class provides a facility similar to list in Python, and the underlying implementation is based on array in C++.

```
#include <vector.h>
```

- Besides including the appropriate library interface, as a client of the **`vector`** class, you are concerned with a different set of issues and need to answer the following questions:
  - How is it possible to specify the type of object (base/element type) contained in a **`vector`**?
  - How does one create an object that is an instance of the **`vector`** class (declaration)?
  - What methods exist in the **`vector`** class to implement its abstract behavior?

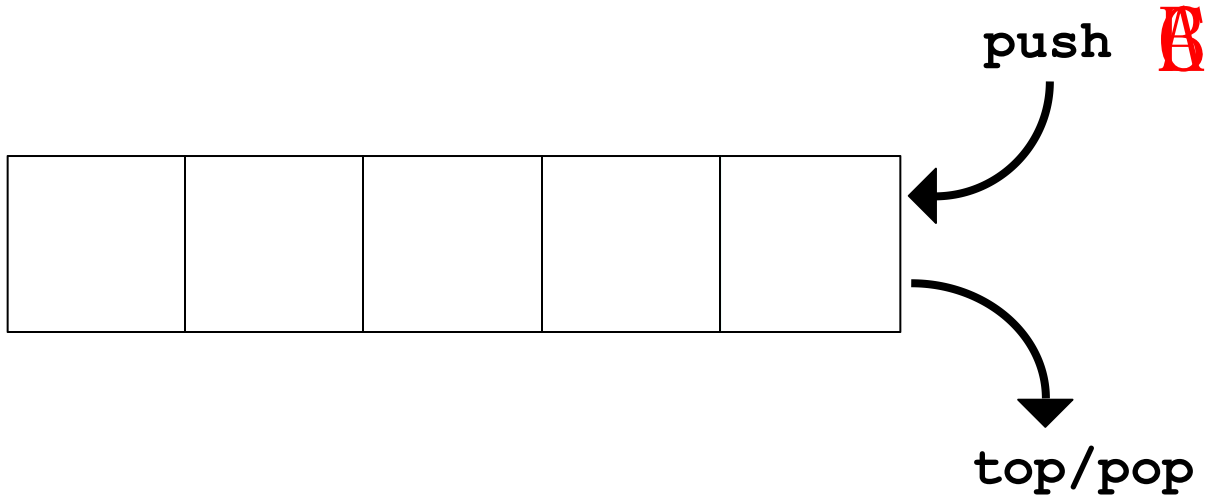
# Declaring a **vector** object

- Instead of using the class name alone, the collection classes require a type parameter that specifies the element type. For example, **vector<int>** represents a vector of integers.

| |
|---|
| **vector<type> vec;** e.g `vector<int> vec;`<br>Initializes an empty vector of the specified element type. |
| **vector<type> vec(n);** e.g `vector<int> vec(5);`<br>Initializes a vector with **n** elements all set to the default value of the type. |
| **vector<type> vec(n, value);** e.g. `vector<int> vec(5,42);`<br>Initializes a vector with **n** elements all set to **value**. |
| `vector<int> vec = {1, 2, 3, 4, 5, 6, 7, 8, 9};//after C++ 11` |

# Stacks

*Stack:*

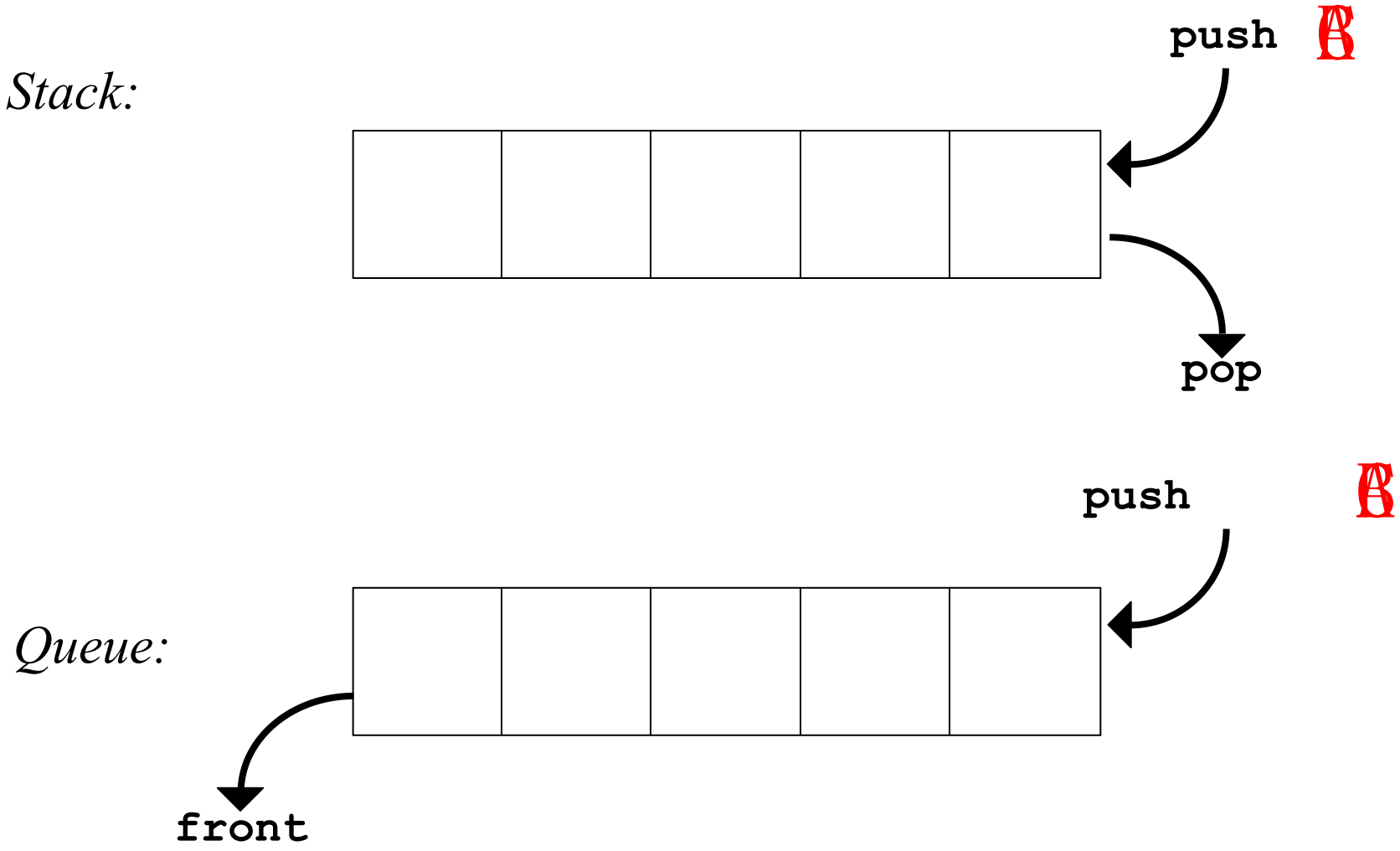



- The ***LIFO*** discipline in a ***stack*** is useful in programming contexts because it reflects the operation of function calls.
- We have used stack in many cases such as DFS graph traversal, Text Editor implementation, Iterative implementation of Maze.

# The **queue<*type*>** Class

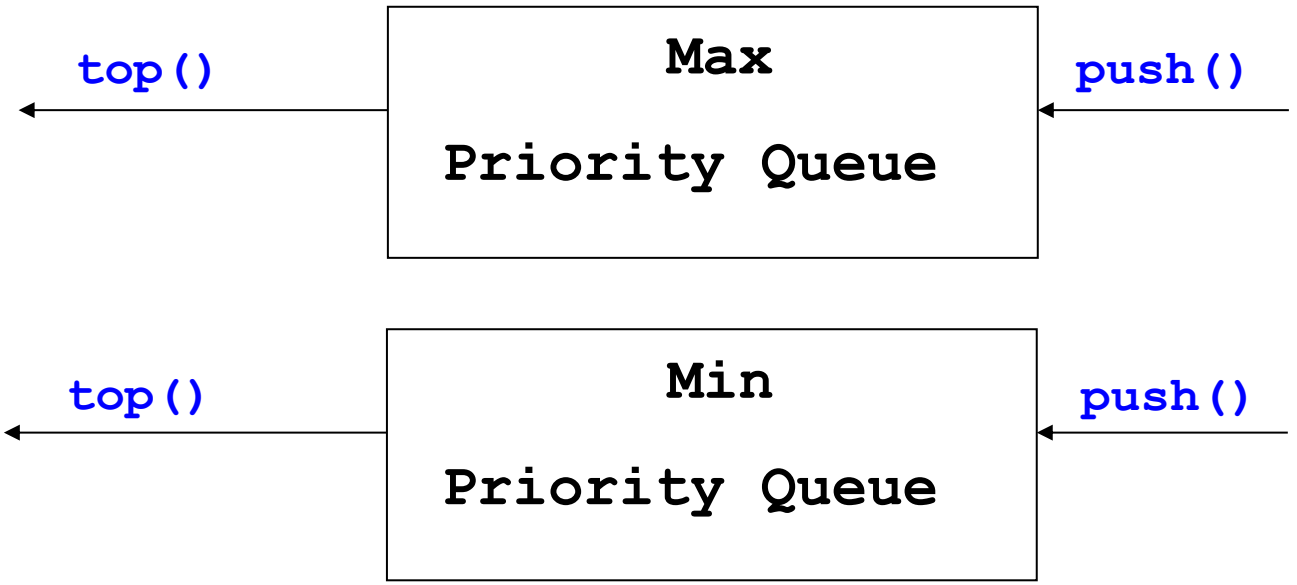
- The ***LIFO*** discipline in a ***stack*** is useful in programming contexts because it reflects the operation of function calls.
- In real-world situations, however, our collective notion of fairness assigns some priority to being first. In programming, the usual phrasing of this ordering strategy is ***First In, First Out (FIFO)***.
- A data structure that stores items using a ***FIFO*** discipline is called a ***queue***. The fundamental operations on a queue, which are analogous to the **push** and **pop** operations for stacks, are called **push()** and **front()**.
- The **push()** operation adds a new element to the end of the queue, which is traditionally called its ***tail***.
- The **front()** operation removes the element at the beginning of the queue, which is called its ***head***.

# Comparing Stacks and Queues



# Priority Queue

- A priority queue is a data structure that supports two basic operations: insert a new item and remove the minimum or maximum item.




Used in finding the shortest path using the Dijkstra's algorithm

# Iterating over a collection

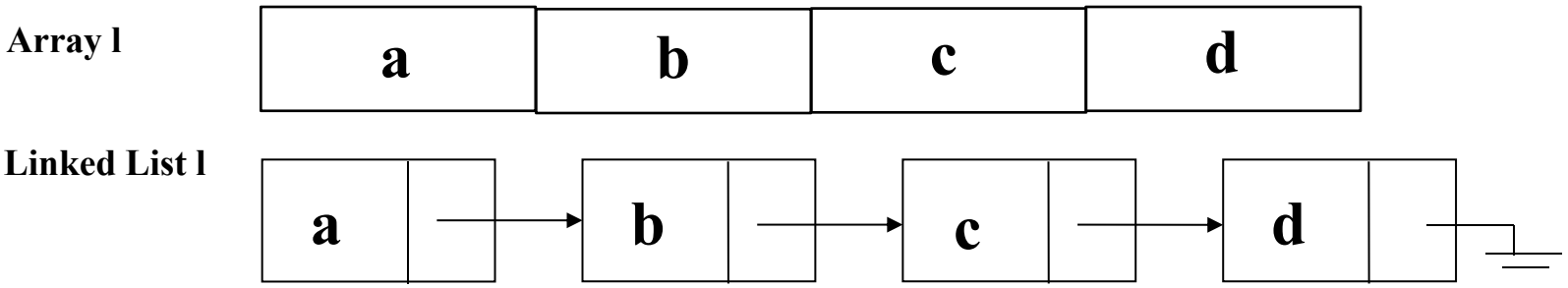
- One of the common operations that clients need to perform when using a collection is to iterate through the elements.
- While it is easy to implement iteration for vectors and grids using **`for`** loops, it is less clear how you would do the same for other collection types. The modern approach to solving this problem is to use a general tool called an ***iterator*** that delivers the elements of the collection, one at a time.
- C++11 uses a ***range-based for statement*** to simplify iterators:

```
for (string key : map) { . . . code to process that key . . . }
```

- ***Range-based for*** (provided since C++ 11) is a way to access ***iterators*** (provided by C++ and implemented by the implementors of the collections).
- The dereference operator (*), pre-inclement operator (++) and inequality operator (!=) are required for a range-based iterator.

# Linked List Basics

- A linked list is made of nodes that are pointing to each other.
- The *array's* features all follow from its strategy of allocating the memory for all its elements in one block of memory.
- *Linked lists* use an entirely different strategy: linked lists allocate memory for each element separately and only when necessary.

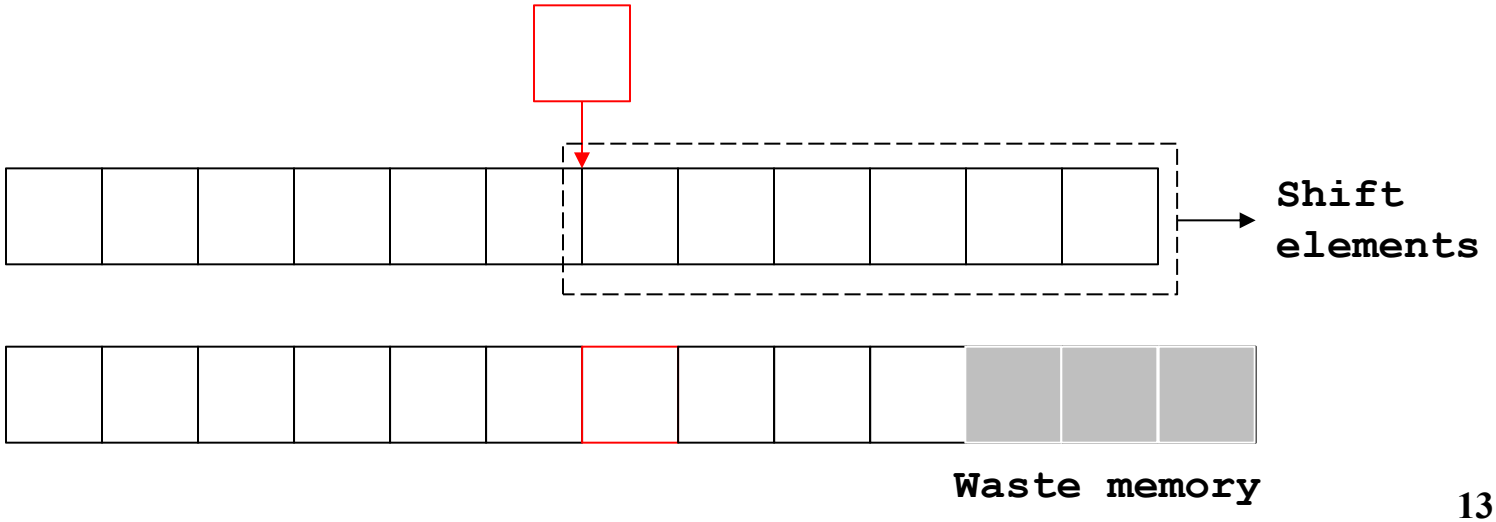


# Linked List Basics

- Linked lists are used to store a collection of information (like arrays)
- A linked list is made of nodes that are pointing to each other
- We know the address of the first node (head) and the last node (tail)
- Other nodes are reached by following the “next” pointers

# Linked List vs. Array

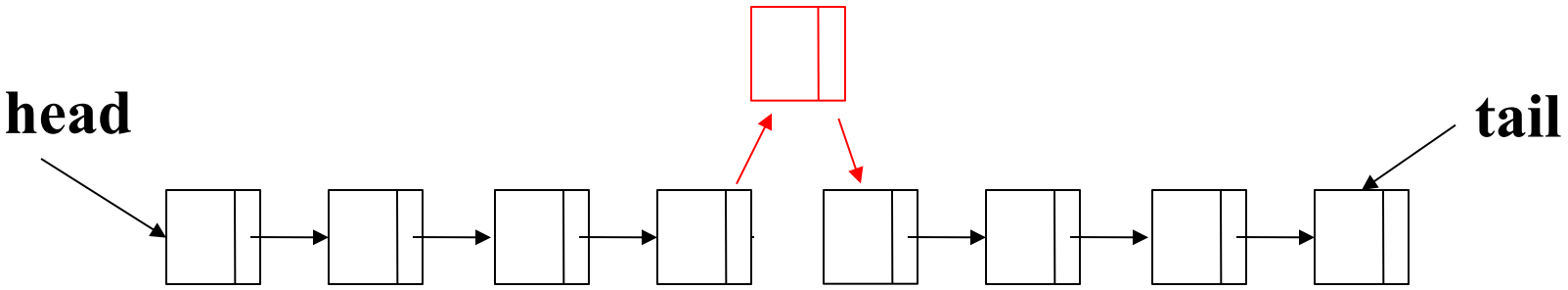
- **Advantages of Arrays**
  - Can directly select any element e.g. data[i]
  - No memory wasted for storing pointers
- **Disadvantages of Arrays:**
  - "Fixed" size (grow or shrink dynamically step-wise block-by-block)
  - Need to shift elements to insert an element to the middle
  - Memory wasted due to unused elements



# Linked List vs. Array

- **Advantages of Linked Lists:**
  - **Dynamic size (can grow and shrink as needed)**
  - **No need to shift elements to insert into the middle**
  - **Size can exactly match the number of elements (no wasted memory)**

- **Disadvantages of Linked Lists**
  - **Cannot directly select any element (need to follow pointers)**
  - **Extra memory usage for storing pointers**




Linked List

# Complexity of the Editor Operations

Efficiency comparison

| | | *array* | *list* | *stack* |
|---|---|---|---|---|
| **F** | **moveCursorForward()** | $O(1)$ | $O(1)$ | $O(1)$ |
| **B** | **moveCursorBackward()** | $O(1)$ | $O(N)$ | $O(1)$ |
| **J** | **moveCursorToStart()** | $O(1)$ | $O(1)$ | $O(N)$ |
| **E** | **moveCursorToEnd()** | $O(1)$ | $O(N)$ | $O(N)$ |
| **I** | **insertCharacter(ch)** | $O(N)$ | $O(1)$ | $O(1)$ |
| **D** | **deleteCharacter()** | $O(N)$ | $O(1)$ | $O(1)$ |

# The **Map<***type, type***>** Class

- A ***map*** is conceptually similar to a dictionary in real life (and in Python), which allows you to look up a word to find its meaning.
- The **Map** class is a generalization of this idea that provides an association between an identifying tag called a ***key*** (e.g., the word in a dictionary) and an associated ***value*** (e.g., the definition of the word in the dictionary), which may be a much larger and more complicated structure.
- Map declaration: **Map<***key type, value type***> map;**
- The type for the keys stored in a **Map** must define a natural ordering, usually through a **less** function and/or **<** operator so that the keys can be compared and ordered.
- E.g., two most commonly used maps:

```
Map<string, string> dictionary;
Map<string, double> symbolTable;
```

# Implementation Strategies for Maps

- There are several strategies you might choose to implement the map operations **get** and **put**. Those strategies include:

  1. ***Linear search***. Keep track of all the key/value pairs in a vector. In this model, both the **get** and **put** operations run in $O(N)$ time.

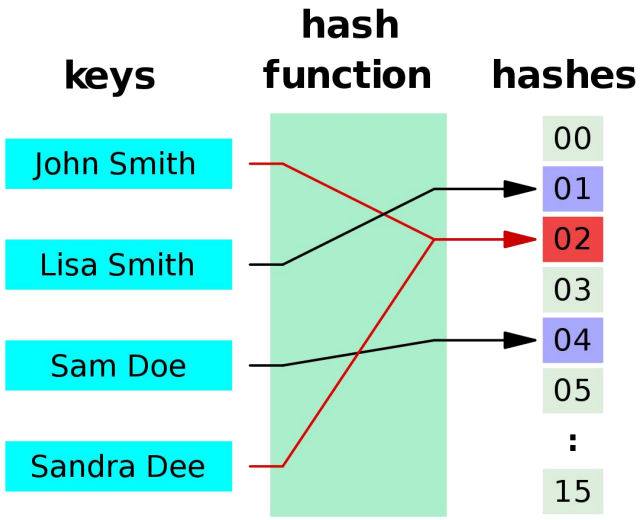
  2. ***Binary search***. If you keep the vector sorted by the key, you can use binary search to find the key. Using this strategy improves the performance of **get** to…?

     $O(\log N)$. But to keep the vector sorted, **put** runs in…?

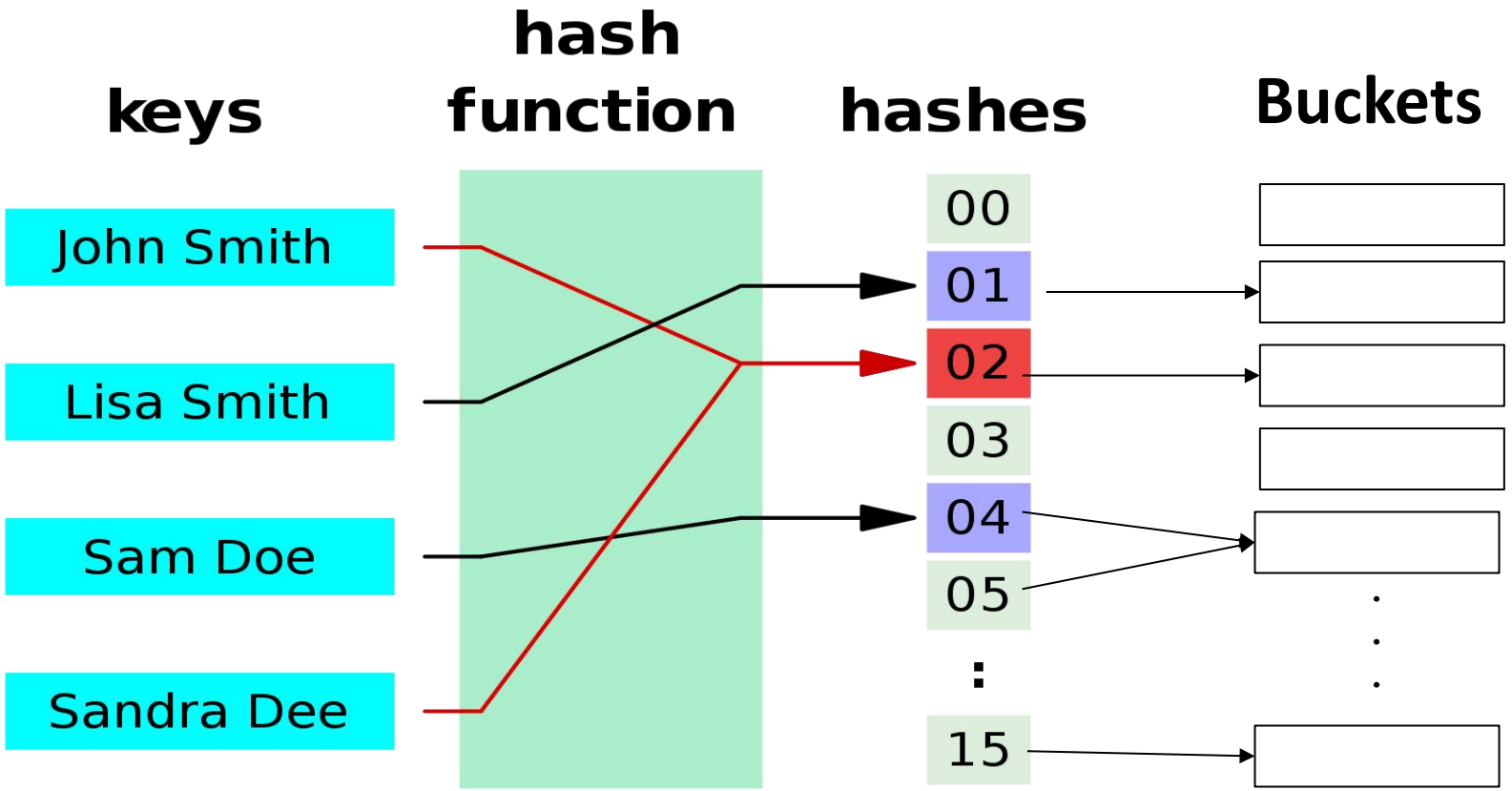
  3. ***Table lookup in a grid***. In this specific example, you can store the state names (two-character code) in a 26 x 26 **Grid<string>** in which the first and second indices correspond to the two letters in the code. Because you can now find any code in a single step, this strategy is $O(1)$, although this performance comes at a cost in memory space (only 50/676 occupied).

# Hash Maps

- The **HashMap** class implemented using the hashing strategy is called a ***hash map*** (sometimes a ***hash table***).
- The implementation requires the existence of a free function (called a ***hash function***) that transforms a key into a non-negative integer (called a ***hash code***), preferably in constant time. The hash code tells the implementation where it should look for a particular key, thereby reducing the search time dramatically.
- The important things to remember about hash codes are:
  1. From a certain hash function, every key has a hash code, and only one hash code.
  2. If two keys are different, they ideally should not, but practically could, have the same hash code (called ***collision***).



# Hashing: from key to bucket




`Hasshes%num_buckets`

# Hash Functions

- To achieve the high level of efficiency that hashing offers, a hash function must have the following two properties:
  1. The function must be inexpensive to compute.
  2. The function should distribute keys as uniformly as possible across the integer range, to minimize ***collisions***.
- Hash functions are extremely subtle and often depend on sophisticated mathematics. Most implementations use techniques similar to those for generating pseudorandom numbers, to ensure that the results are hard to predict, hence are unlikely to exhibit any higher level of collision than one would expect by chance (e.g., thumb index collisions are not by chance).
- Fortunately, the correctness of the algorithm is not affected by the collision rate. Implementations that use poorly designed hash functions run more slowly but nonetheless continue to give correct results.

# Achieving $O(1)$ Performance

- The simulation on the previous slide uses only seven buckets to emphasize what happens when collisions occur: the smaller the number of buckets, the more likely collisions become.
- In practice, the implementation of **Map<string, string>** would use a much larger value for **nBuckets** to minimize the opportunity for collisions. If the number of buckets is considerably larger than the number of keys, most of the bucket chains will either be empty or contain exactly one key/value pair.
- The ratio of the number of keys to the number of buckets is called the ***load factor*** of the map. Because a map achieves $O(1)$ performance only if the load factor ($N_{entries}/N_{buckets}$) is small, the library implementation of **HashMap** increases the number of buckets when the map becomes too full. This process is called ***rehashing***.
- $O(1)$ is achieved on average, and the worst case is still $O(N)$.

# Set Operations

- A ***set*** is an unordered collection of distinct values.
- The fundamental set operation is ***membership*** ($\in$).

| | |
|---|---|
| $3 \in$ **primes** | $3 \notin$ **evens** |
| *red* $\in$ **primary** | *red* $\notin$ **secondary** |
| $-1 \in$ **Z** | $-1 \notin$ **N** |

- The ***union*** of two sets $A$ and $B$ ($A \cup B$) consists of all elements in either $A$ or $B$ or both.
- The ***intersection*** of $A$ and $B$ ($A \cap B$) consists of all elements in both $A$ and $B$.
- The ***set difference*** of $A$ and $B$ ($A - B$) consists of all elements in $A$ but not in $B$.
- Set $A$ is a ***subset*** of $B$ ($A \subseteq B$) if all elements in $A$ are also in $B$.
- Sets $A$ and $B$ are ***equal*** ($A = B$) if they have the same elements.

# Implementing Sets

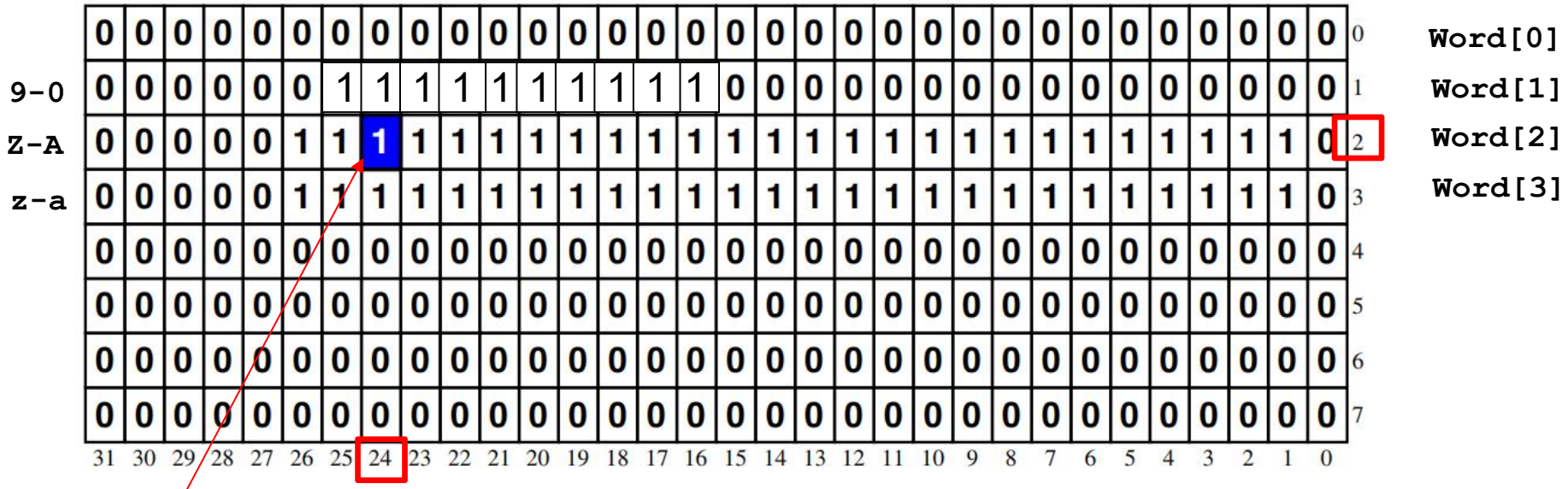
Different implementations:

- ***Pointer based*** implementation, the very basic implementation
- ***Hash tables***. Sets implemented as hash tables are extremely efficient, offering average $O(1)$ performance for adding a new element or testing for membership. The primary disadvantage is that hash tables do not support ordered iteration
- ***Balanced binary trees***. Sets implemented using balanced binary trees offer $O(\log N)$ performance on the fundamental operations, but do make it possible to write an ordered iterator.
- The **`set`** class in standard C++ uses the BST approach. The **`unordered_set`** class (from C++11) uses the Hash Tables.
- One of the implications of using the BST representation is that the underlying value type must support the comparison operators **==** and **<**.

# Character Sets

- The key insight needed to make efficient ***character sets*** (or, equivalently, ***sets of small integers***) is that you can represent the inclusion or exclusion of a character (a set element) using a single bit. If the bit is a 1, then that element is in the set; if it is a 0, it is not in the set.
- You can tell what character value you're talking about by creating what is essentially an array of bits, with one bit for each of the ASCII codes. That array is called a ***characteristic vector/indicator vector***.
- What makes this representation so efficient is that you can pack the bits for a characteristic vector into a small number of words inside the machine and then operate on the bits in large chunks.
- The efficiency gain is enormous. Using this strategy, most set operations can be implemented in just a few instructions.

# Bit Vectors and Character Sets




```
X = char(k), k=88
```

```
row = k/32 = 2  (when k = 88)
Col = k%32 = 24 (when k = 88)
```

```
A  = 65, row = 2, col =1
a  = 97, row = 3, col =1
```

# Implementing Characteristic Vectors

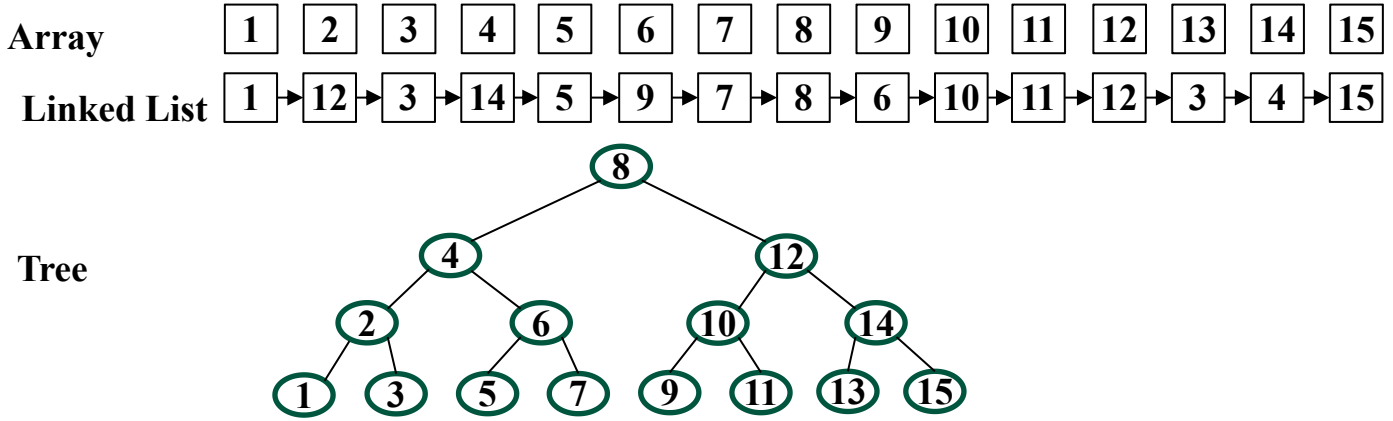
```
const int BITS_PER_BYTE = 8;
const int BITS_PER_LONG = BITS_PER_BYTE * sizeof(long);
const int CVEC_WORDS = (RANGE_SIZE + BITS_PER_LONG - 1) / BITS_PER_LONG;
struct CharacteristicVector {
   unsigned long words[CVEC_WORDS];
};
unsigned long createMask(int k) {
   return unsigned long(1) << k % BITS_PER_LONG;
}
bool testBit(CharacteristicVector & cv, int k) {
    if (k < 0 || k >= RANGE_SIZE) {
       error("testBit: Bit index is out of range");
    }
    return cv.words[k / BITS_PER_LONG] & createMask(k);
}
void setBit(CharacteristicVector & cv, int k) {
   if (k < 0 || k >= RANGE_SIZE) {
      error("setBit: Bit index is out of range");
   }
   cv.words[k / BITS_PER_LONG] |= createMask(k);
}
void clearBit(CharacteristicVector & cv, int k) {
   if (k < 0 || k >= RANGE_SIZE) {
      error("setBit: Bit index is out of range");
   }
   cv.words[k / BITS_PER_LONG] &= ~createMask(k);
}
```

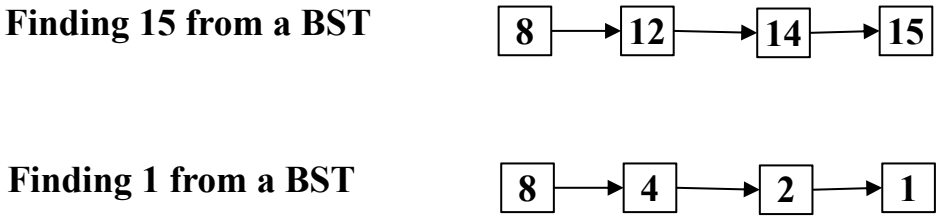
*column index - note here the bits are stored **from right to left** to simplify the calculation.*

*row index*

# The motivation behind Binary Search Trees

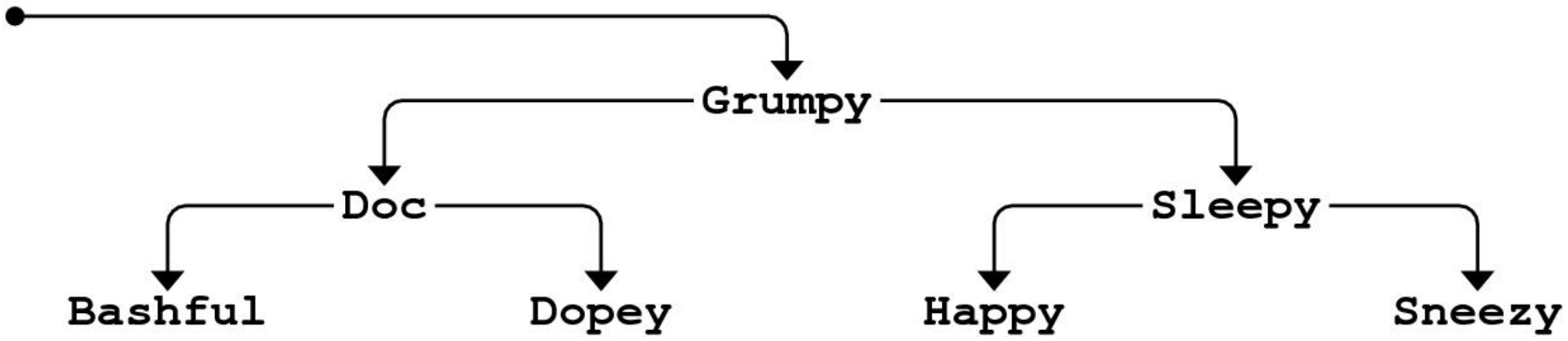
- The idea is to combine the linked list data structure (for easier insertion/deletion) and the binary search algorithm (for faster search).



# Binary Search Trees

- The ***binary search tree*** (or ***BST*** for short):
  - Each node in a BST has exactly two subtrees: a ***left subtree*** that contains all the nodes that less than the current node and a ***right subtree*** that contains all the nodes that larger than it. Either or both of these subtrees may be **`NULL`**.
  - Both left subtree and right subtree are also BSTs.
- The classic example of a binary search tree uses the names from Walt Disney's *Snow White and the Seven Dwarves*:



# A Simple BST Implementation

- To get a sense of how binary search trees work, it is useful to start with a simple design in which keys are always strings.
- Each node in the tree is then a structure containing a key and two subtrees, each of which is either **NULL** or a pointer to some other node. This design suggests the following definition:

```
struct BSTNode {
   string key;
   BSTNode *left, *right;
};
```

- The code for finding a node in a tree begins by comparing the desired key with the key in the root node. If the strings match, you've found the correct node; if not, you simply call yourself recursively on the left or right subtree depending on whether the key you want comes before or after the current one.

# A Simple BST Implementation

```
/*
 * Function: insertNode
 * Usage: insertNode(t, key);
 * --------------------------
 * Inserts the specified key at the appropriate location in the
 * binary search tree rooted at t.  Note that t must be passed
 * by reference, since it is possible to change the root.
 */

void insertNode(Node* & t, string key) {
   if (t == NULL) {
      t = new Node;
      t->key = key;
      t->left = t->right = NULL;
      return;
   }
   if (key == t->key) return;
   if (key < t->key) {
      insertNode(t->left, key);
   } else {
      insertNode(t->right, key);
   }
}
```

*Call a pointer by reference. Can you use call by pointer here?*

# A Simple BST Implementation

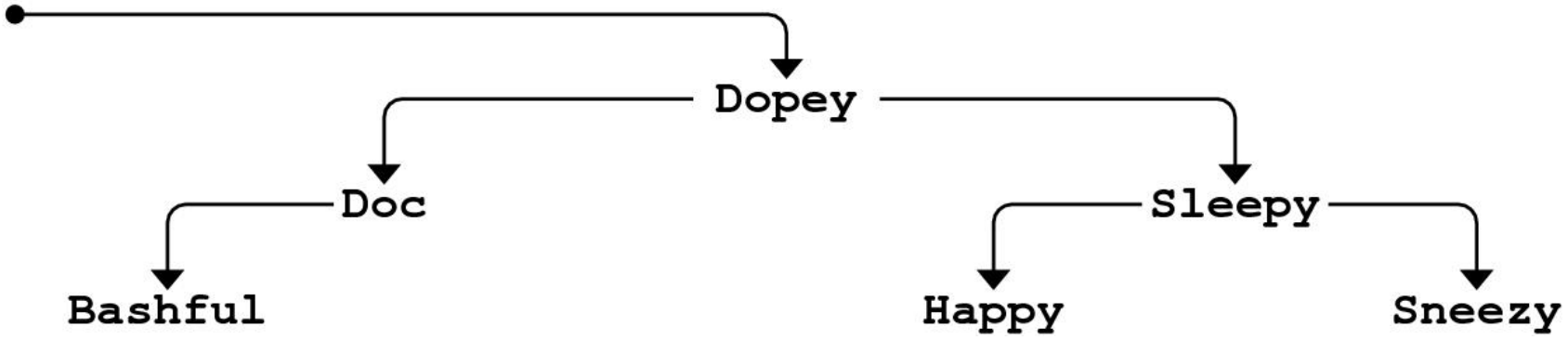
```
/*
 * Function: insertNode
 * Usage: insertNode(t, key);
 * --------------------------
 * Inserts the specified key at the appropriate location in the
 * binary search tree rooted at t.  Note that t must be passed
 * by reference, since it is possible to change the root.
 */

void insertNode(Node* & t, string key) {
   if (t == NULL) {
      t = new Node;
      t->key = key;
      t->left = t->right = NULL;
      return;
   }
   if (key == t->key) return;
   if (key < t->key) {
      insertNode(t->left, key);
   } else {
      insertNode(t->right, key);
   }
}
```

*Call a pointer by reference. Can you use call by pointer here?*

# Removing Nodes in Binary Search Trees

- For a leaf node (with no children), replace the pointer to the node with a NULL pointer;
- If either child of the node you want to remove is NULL, replace it with its non-NULL child;
- If you try to remove a node with both a left and a right child, replace it with the rightmost node in the left subtree(max) (predecessor) or the leftmost node in the right subtree(min) (successor). Additional operations might be needed.



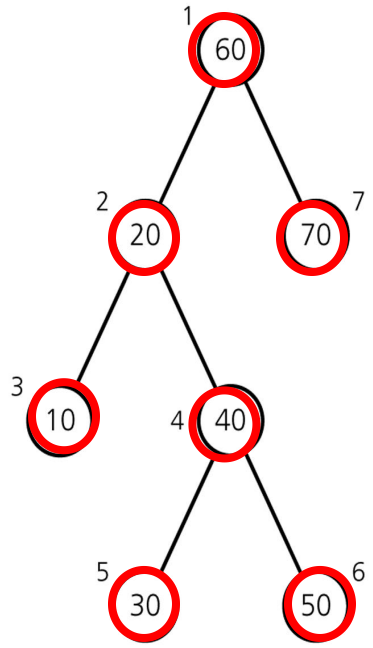
# Order of Operations on BSTs

| | Balanced BST | Unbalanced BST | | |
|---|---|---|---|---|
| Operation | Average case | Worst case | Array | LinkedList |
| Retrieval | O(log n) | O(n) | O(1) | O(n) |
| Insertion | O(log n) | O(n) | O(n) | O(1) |
| Deletion | O(log n) | O(n) | O(n) | O(1) |
| Traversal | O(n) | O(n) | O(n) | O(n) |

When $\log_2 n = 18$, $n = 2^{18} = 262144$

O(log 262144) =18 operations, O(262144) = 262144 operations

# Binary Tree Traversals



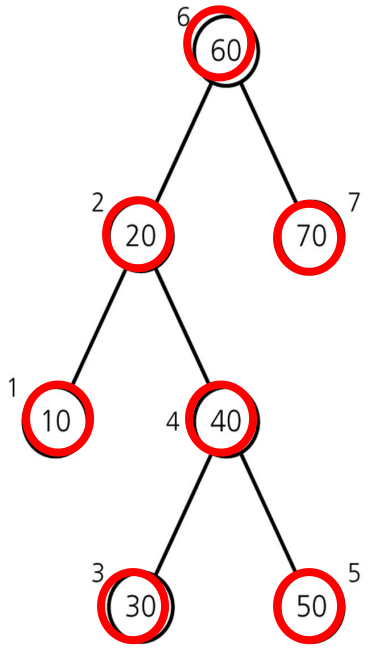

(a) Preorder: 60, 20, 10, 40, 30, 50, 70

(Numbers beside nodes indicate traversal order.)

```
void preorder(Node * p) {
  if (p !=NULL) {
       cout << p->key << endl;
       preorder(p->left_child);
       preorder(p->right_child);
    }
}
```

**Preorder Traversal (Root-Left-Right)**

# Binary Tree Traversals



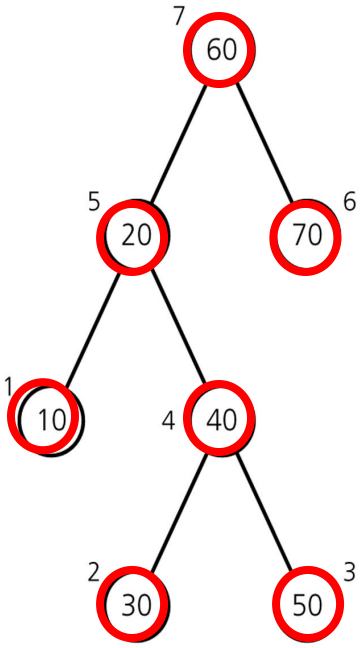

(b) Inorder: 10, 20, 30, 40, 50, 60, 70

```
void inorder(Node *p)
{
   if (p !=NULL) {
        inorder(p->left_child);
        cout << p->key << endl;
        inorder(p->right_child);
   }
}
```

**Inorder Traversal (Left-Root-Right)**

# Binary Tree Traversals

```
void postorder(Node *p)
{
   if (p !=NULL) {
        postorder(p->left_child);
        postorder(p->right_child);
        cout << p->key << endl;
   }
}
```




(c) Postorder: 10, 30, 50, 40, 20, 70, 60

**Postorder Traversal (Left-Right-root)**